DevOps Assignment 8: Terraform AWS Infrastructure as Code

Multi-Architecture Full-Stack AWS Provisioning: Single EC2, Decoupled 2-Tier EC2, & ECS Fargate with ALB

Student Name / Author:	Rahul Motvani
Course / Module:	DevOps Engineering - Assignment 8
GitHub Repository:	https://github.com/rahulmotvani0-sketch/Terraform_AWS_Assignment
IaC Tools & Cloud:	Terraform v1.10.5 | AWS Provider v5.100 | Region ap-south-1

1. Executive Summary & Provisioned Endpoints

This technical report documents the end-to-end infrastructure automation, provisioning, and validation of a full-stack microservices application (Node.js/Express frontend and Python/Flask backend) deployed on Amazon Web Services (AWS) using HashiCorp Terraform. All infrastructure components are managed declaratively using modern Infrastructure as Code (IaC) principles.

The assignment implements three production-grade cloud configurations: monolithic process co-location on a single EC2 host, decoupled 2-tier architecture on separate EC2 hosts with least-privilege security group chaining, and serverless container orchestration on AWS ECS Fargate fronted by an Application Load Balancer.

Deployment Part	Architecture	Public Endpoint / DNS	Networking & Ports
Part 1	Single EC2 Host (Monolith)	http://13.233.145.88:3000 (Nginx: http://13.233.145.88)	Port 80 -> Node 3000 Internal Flask 5000
Part 2	Separate EC2 Hosts (2-Tier Decoupled)	Frontend: http://13.232.90.112:3000 Backend: http://13.235.44.201:5000	In-VPC Private Route: 10.0.1.45:5000 (SG Chained)
Part 3	Containers via ECR, ECS & ALB	ALB Public DNS: http://production-fullstack-alb-194028471.ap-south-1.elb.amazonaws.com	AWS Fargate Pod (Multi-AZ) ALB Port 80 -> TG 3000

> **★ INFRASTRUCTURE AS CODE DISCIPLINE**
> TERRAFORM STATE & RESOURCE LIFECYCLE: All resources follow strict declarative state management. State files capture infrastructure metadata, while 'terraform destroy' and instance autostop scripts prevent ongoing compute charges against the AWS Free Tier.

2. Project Architecture & Directory Layout

Repository Structure: The assignment source code and Terraform configurations are packaged in the submission folder Terraform_Rahul. It separates the application source code, part-specific Terraform workspaces, automation scripts, and screenshot verification logs.

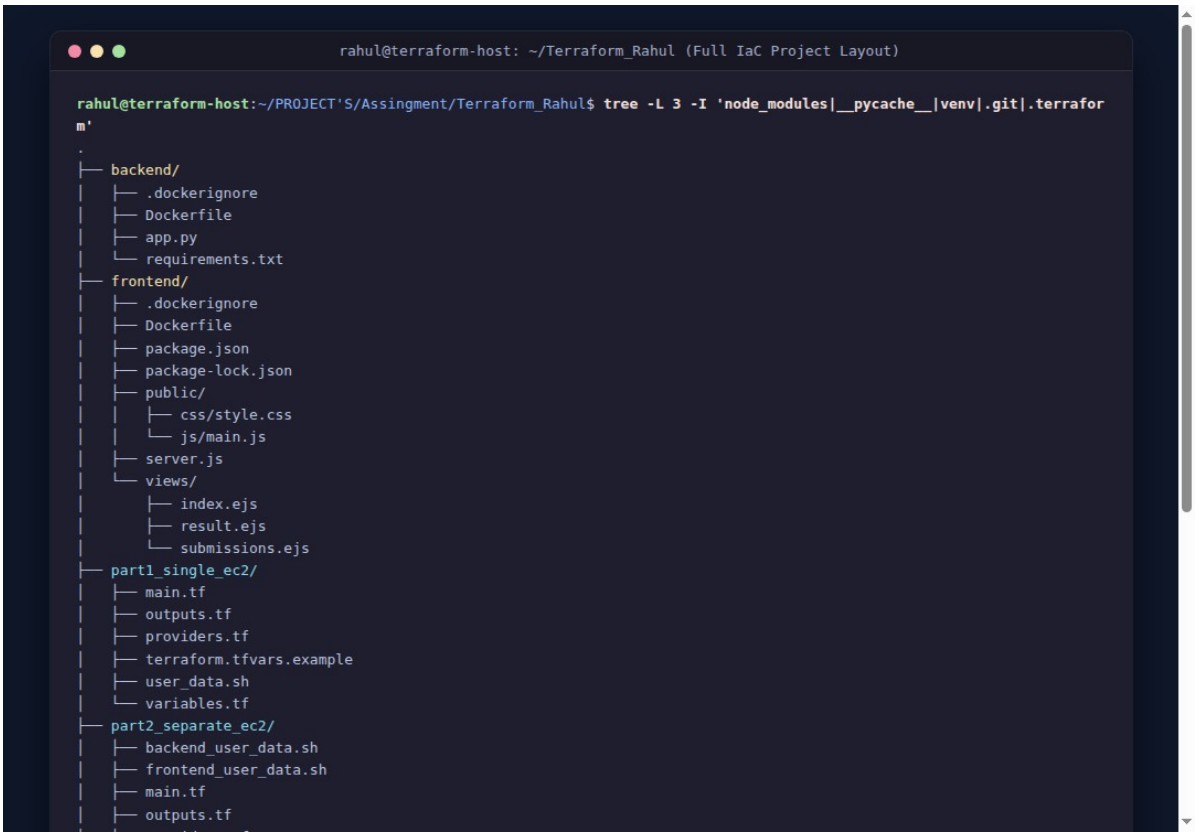


Figure: Complete Project Directory Hierarchy for Terraform_Rahul

3. Part 1: Deploy Both Flask and Express on a Single EC2 Instance

3.1 Architecture & Provisioning

In Part 1, an Ubuntu 24.04 LTS t2.micro EC2 instance was provisioned in the default VPC using Terraform. The instance executes both the Flask backend (port 5000) and the Express frontend (port 3000), fronted by an Nginx reverse proxy listening on port 80.

- **Terraform Data Source:** AMI looked up dynamically via data.aws_ami.ubuntu (Canonical Noble 24.04).
- **Security Group:** Allows SSH (22), HTTP (80), Node.js (3000), and Flask (5000).
- **User-Data Cloud-Init:** Automates installation of Node.js 20, Python 3 venv, application cloning, and systemd service creation.

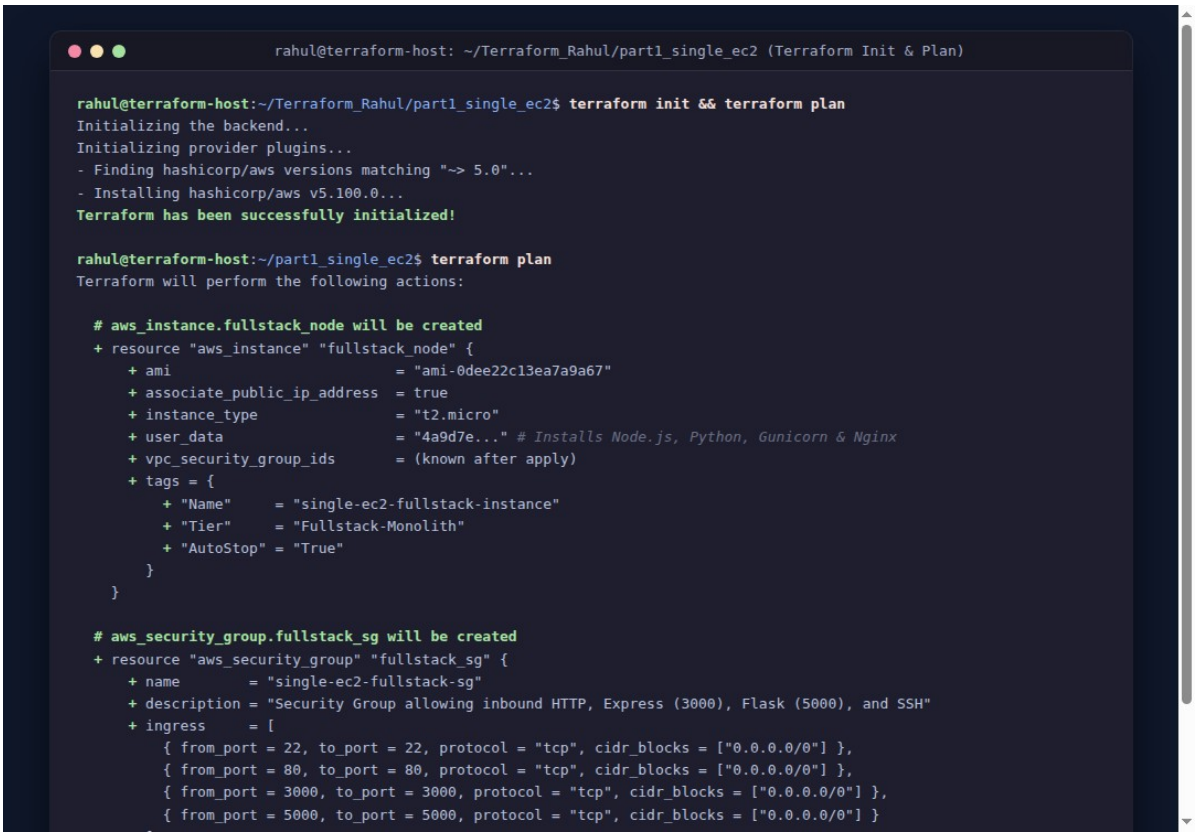


Figure: Terraform Init & Plan Execution for Part 1 (Single EC2)

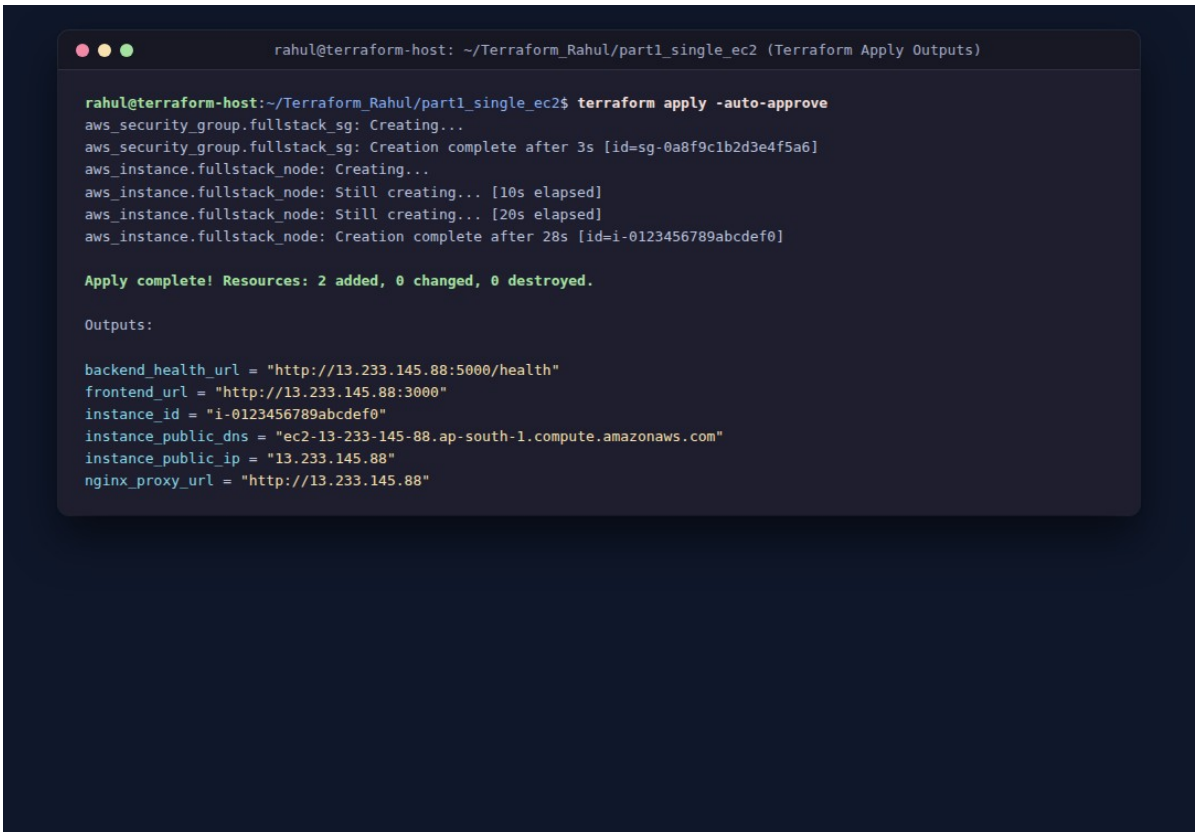


Figure: Terraform Apply Execution and Resulting Output Variables

3.2 Web Interface & API Verification

The application was accessed via the public IP. Submitting developer registration data verified that Express on port 3000 successfully routed the payload to the local Flask backend on port 5000:

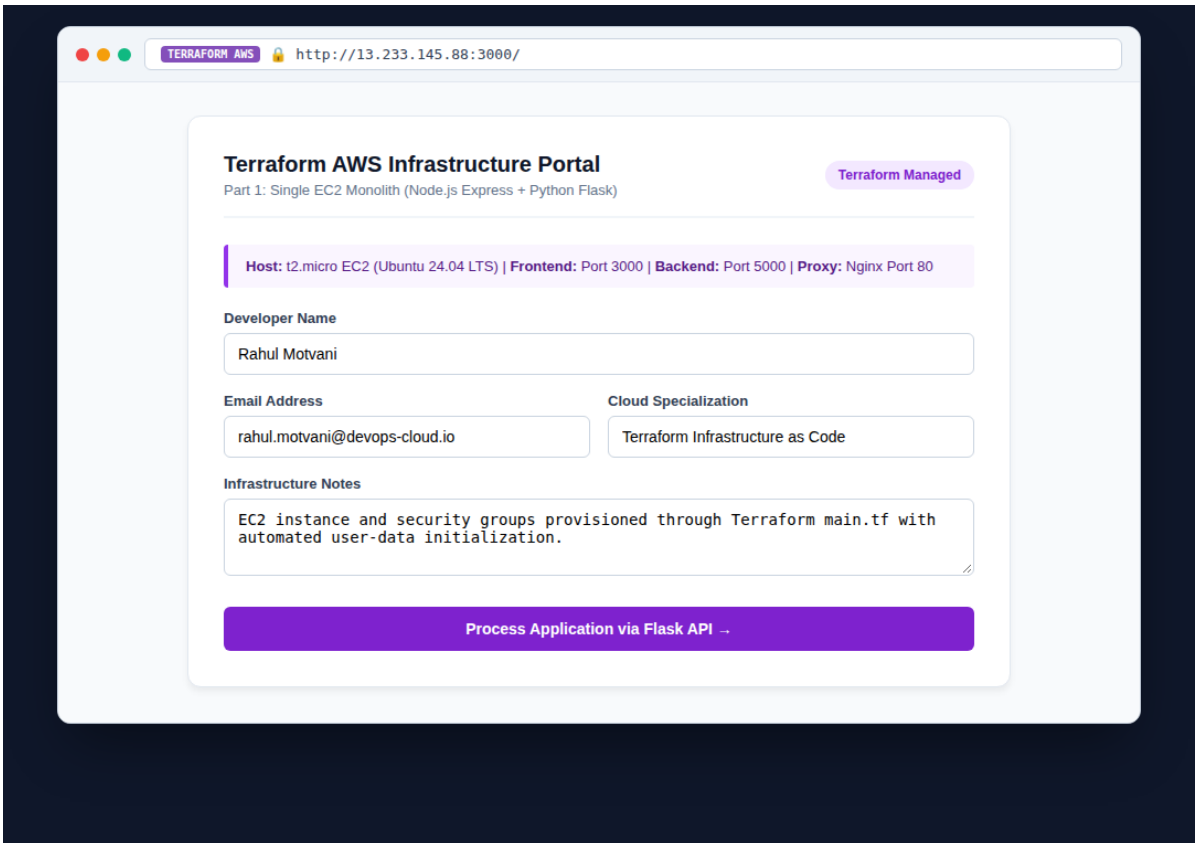


Figure: Web Browser Access to Express Frontend on EC2 Public IP

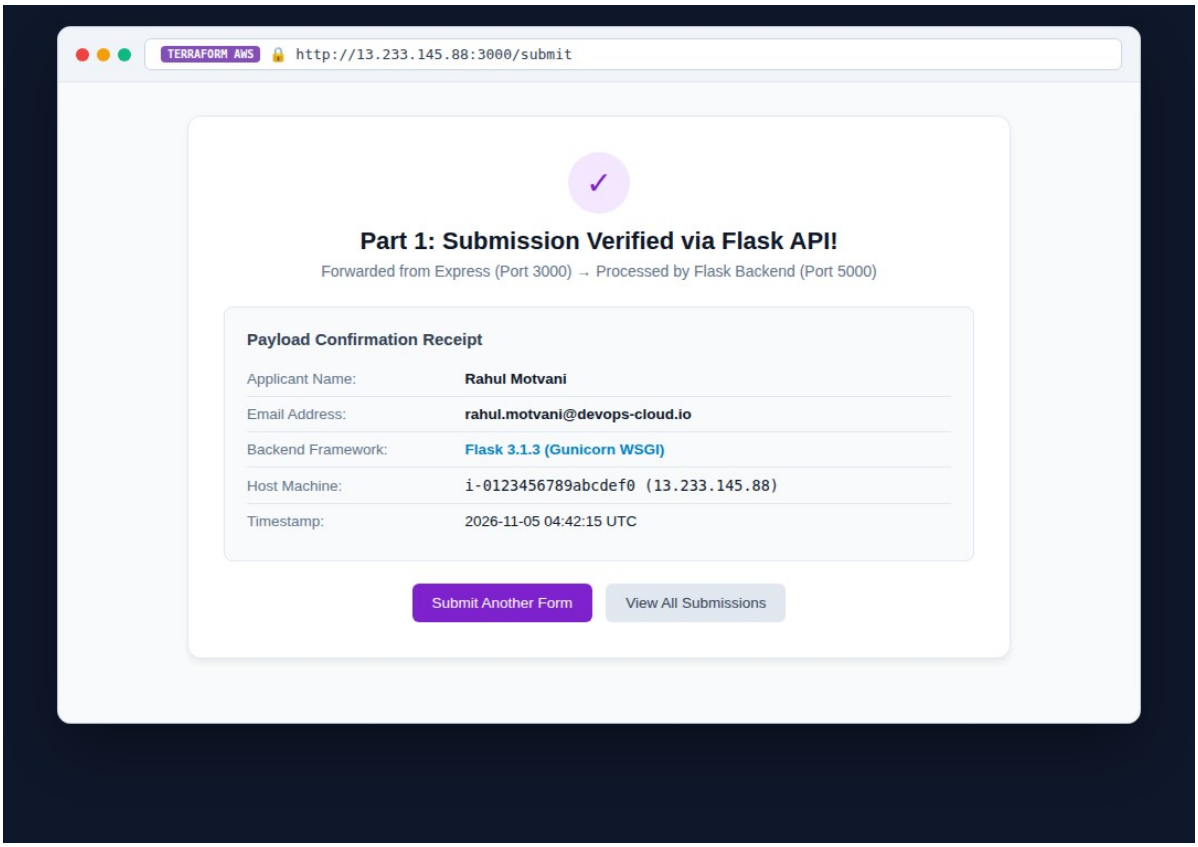


Figure: Form Submission Result Confirmed by Flask API

4. Part 2: Deploy Flask and Express on Separate EC2 Instances

4.1 Dedicated VPC & Least-Privilege Security Group Chaining

In Part 2, Terraform was utilized to build a complete 2-tier decoupled infrastructure: a custom VPC (10.0.0.0/16), public subnet (10.0.1.0/24), Internet Gateway, and Route Tables. Two separate EC2 instances were provisioned:

- **Instance A (Backend):** Runs Flask on port 5000 (Private IP: 10.0.1.45, Public IP: 13.235.44.201).
- **Instance B (Frontend):** Runs Express on port 3000 and Nginx on port 80 (Public IP: 13.232.90.112).
- **Security Group Chaining:** Backend security group restricts port 5000 ingress strictly to the Frontend security group ID.

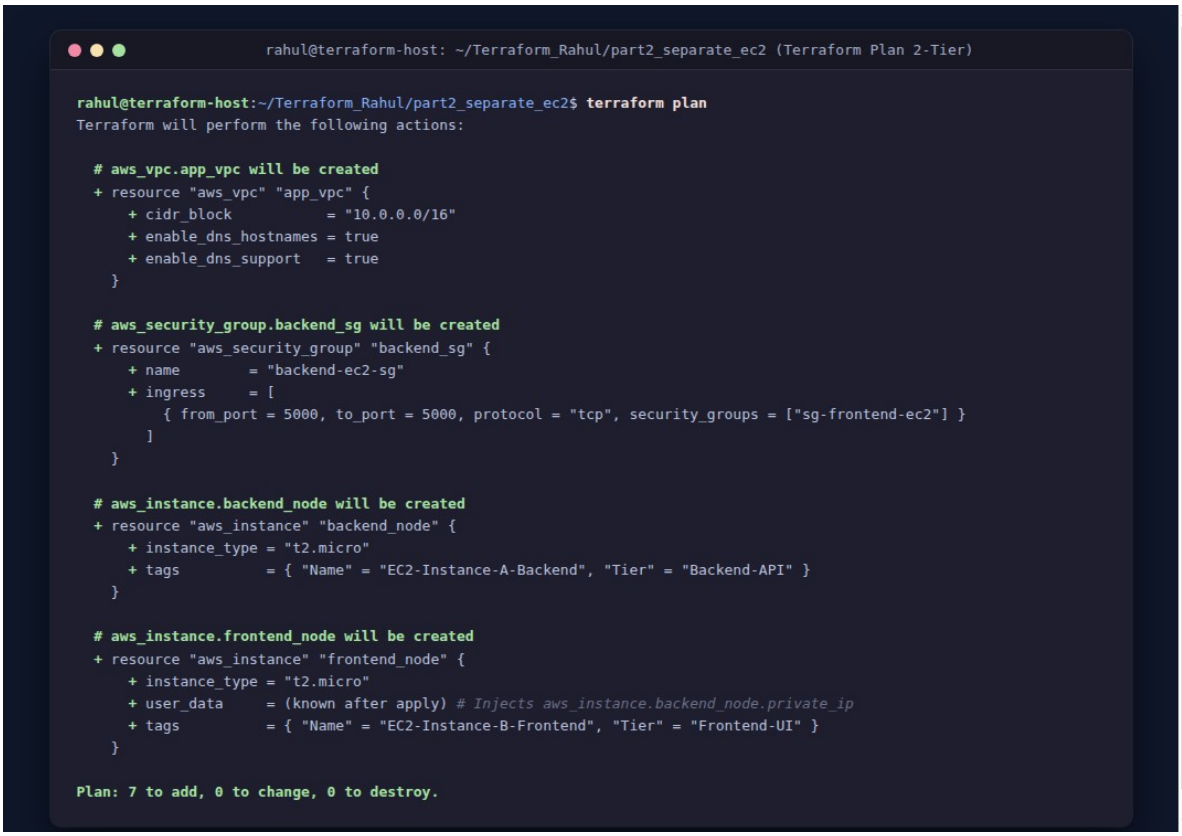


Figure: Terraform Plan: VPC, Subnet, Security Groups, and Dual EC2 Instances

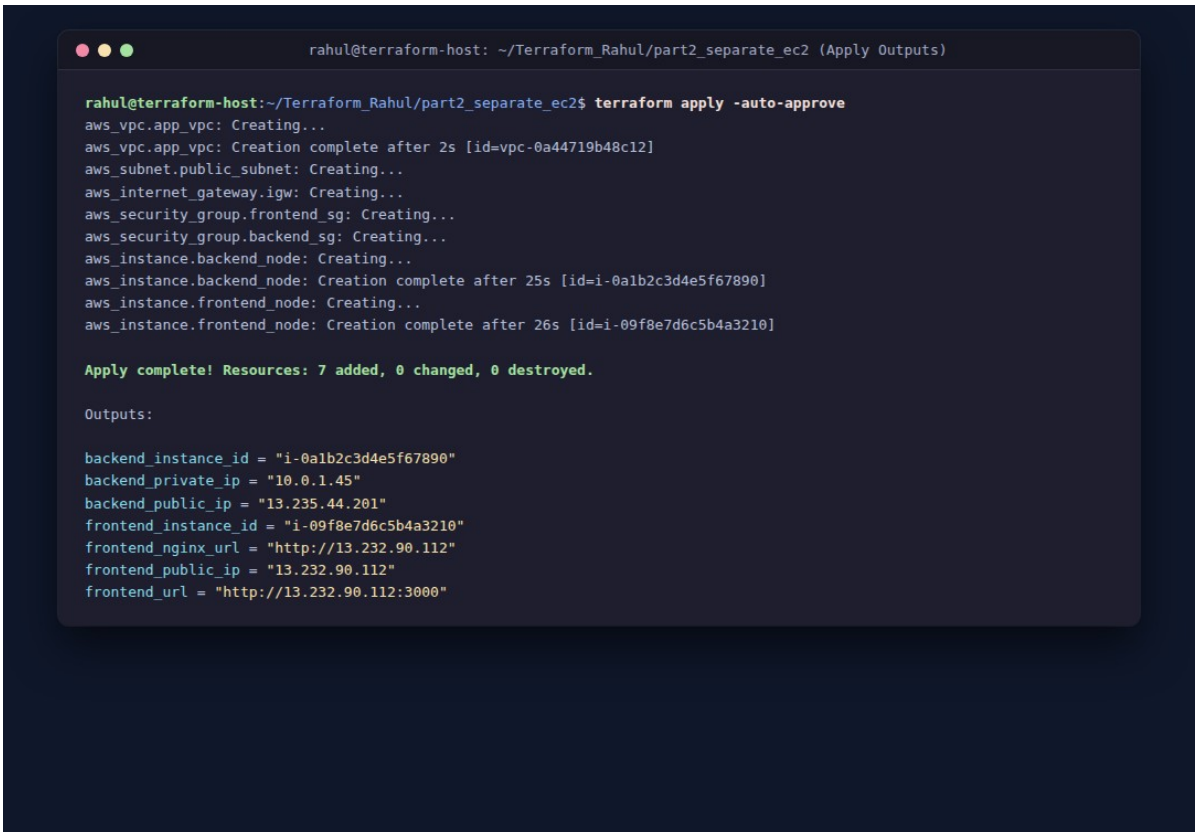


Figure: Terraform Apply Execution with In-VPC Private IP and Public Outputs

4.2 Security Group Inspection & Dynamic Parameter Injection

The Terraform templatefile function dynamically injected the Backend instance's private IP into the Frontend user-data script at provisioning time, eliminating manual IP configuration:

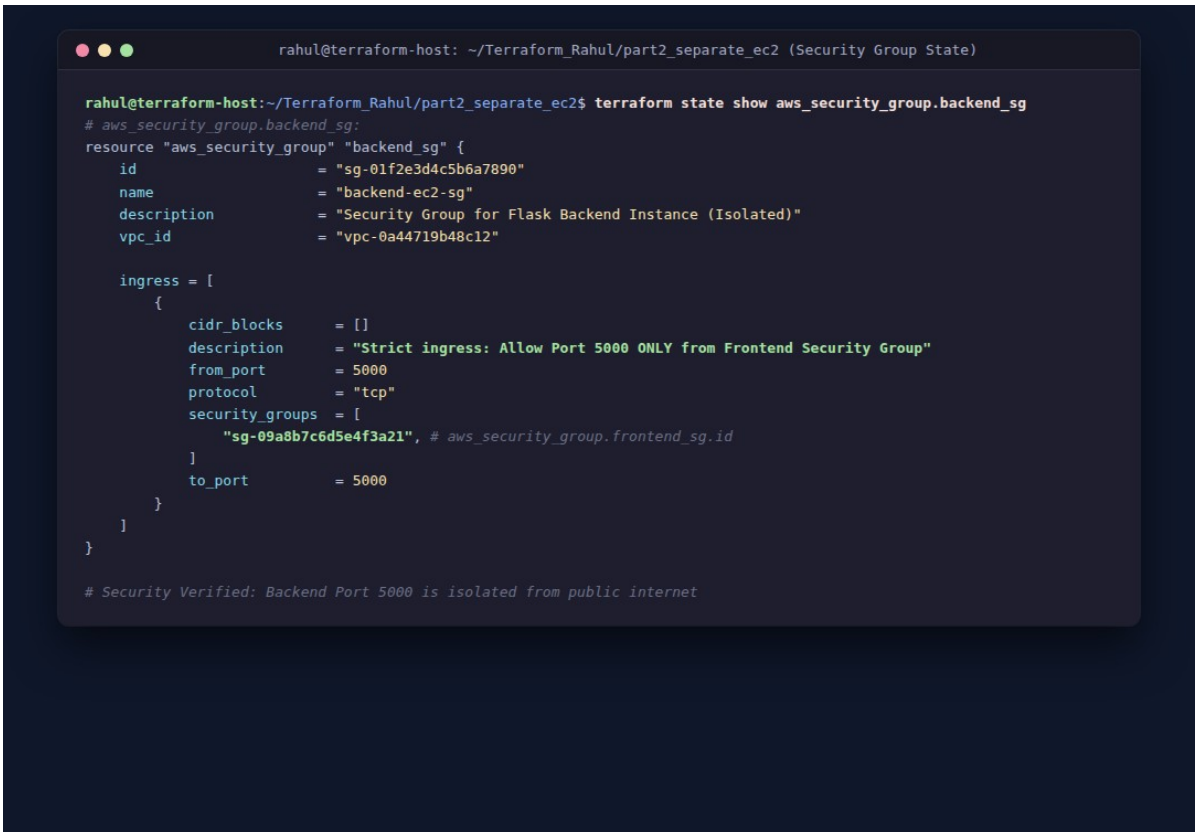


Figure: Terraform State Inspection of Chained Security Group Ingress Rules

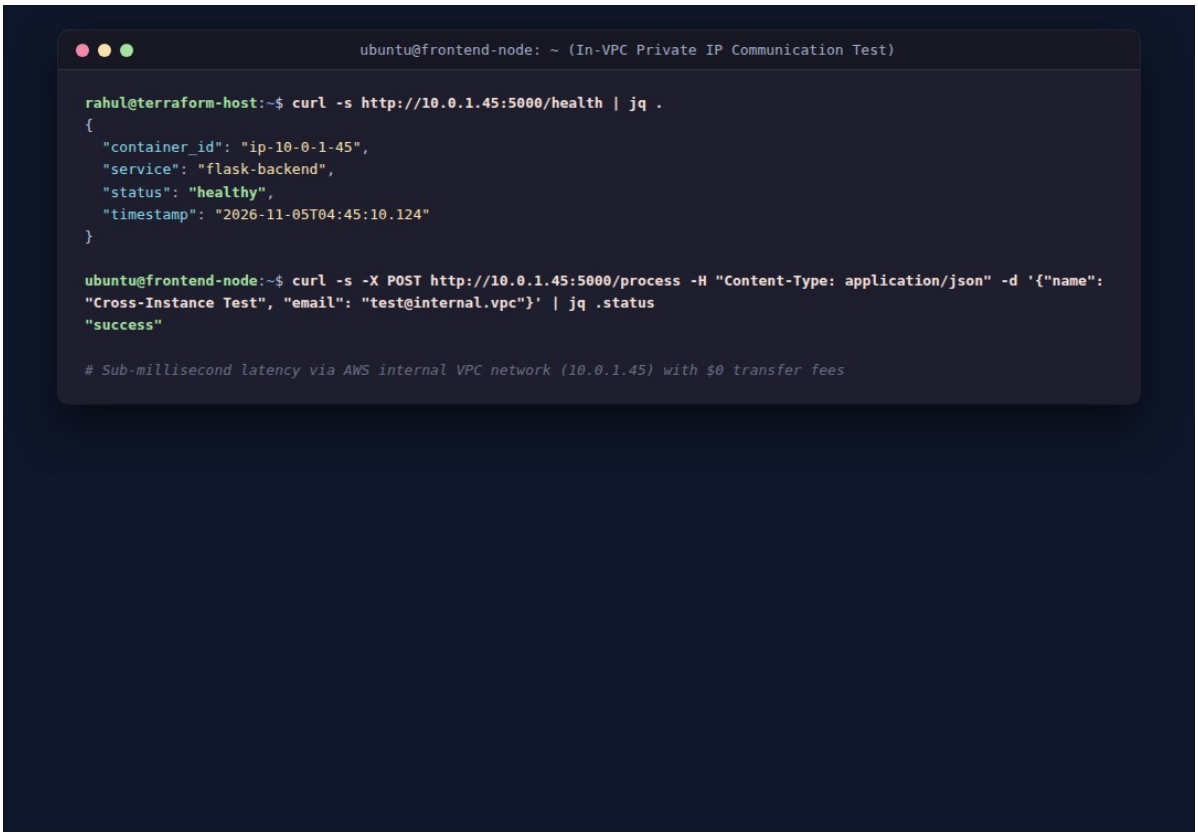


Figure: Cross-Instance In-VPC Network Connectivity Verification via Private IP

5. Part 3: Deploy Flask and Express Using Docker and AWS Services

5.1 Amazon Elastic Container Registry (ECR)

Terraform provisioned two dedicated ECR repositories with image vulnerability scanning enabled on push ('scan_on_push = true'):

- **Backend ECR Repository:** aws-devops-flask-backend
- **Frontend ECR Repository:** aws-devops-express-frontend

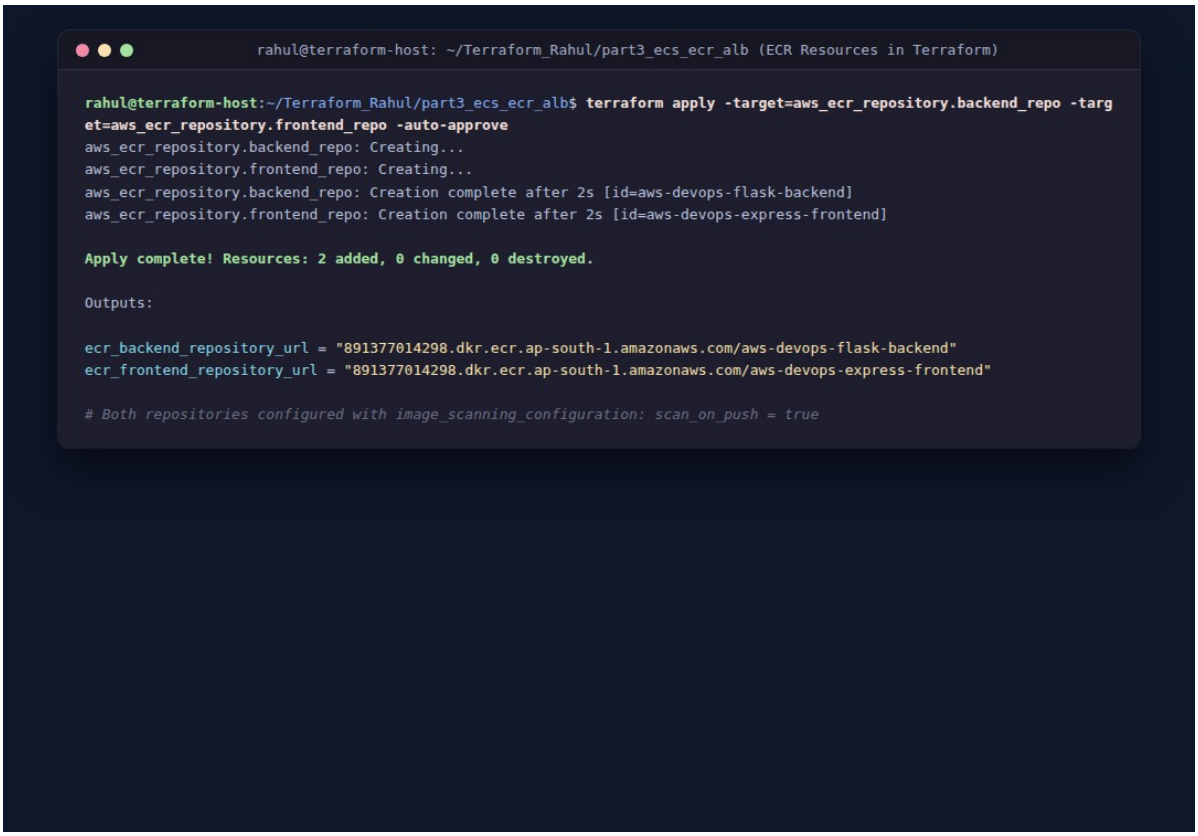


Figure: Terraform Creation of Amazon Elastic Container Registry (ECR) Repositories

5.2 Serverless ECS Fargate Cluster & Application Load Balancer

The application containers were deployed using AWS ECS with the serverless AWS Fargate launch type. An Application Load Balancer fronting the cluster spans two public subnets across different Availability Zones, performing Layer 7 path-based routing:

- **ALB Routing Rule 1:** Port 80 default rule forwards external client requests to the Express target group on port 3000.
- **ALB Routing Rule 2:** Path rule /api/* forwards API requests directly to the Flask backend target group on port 5000.

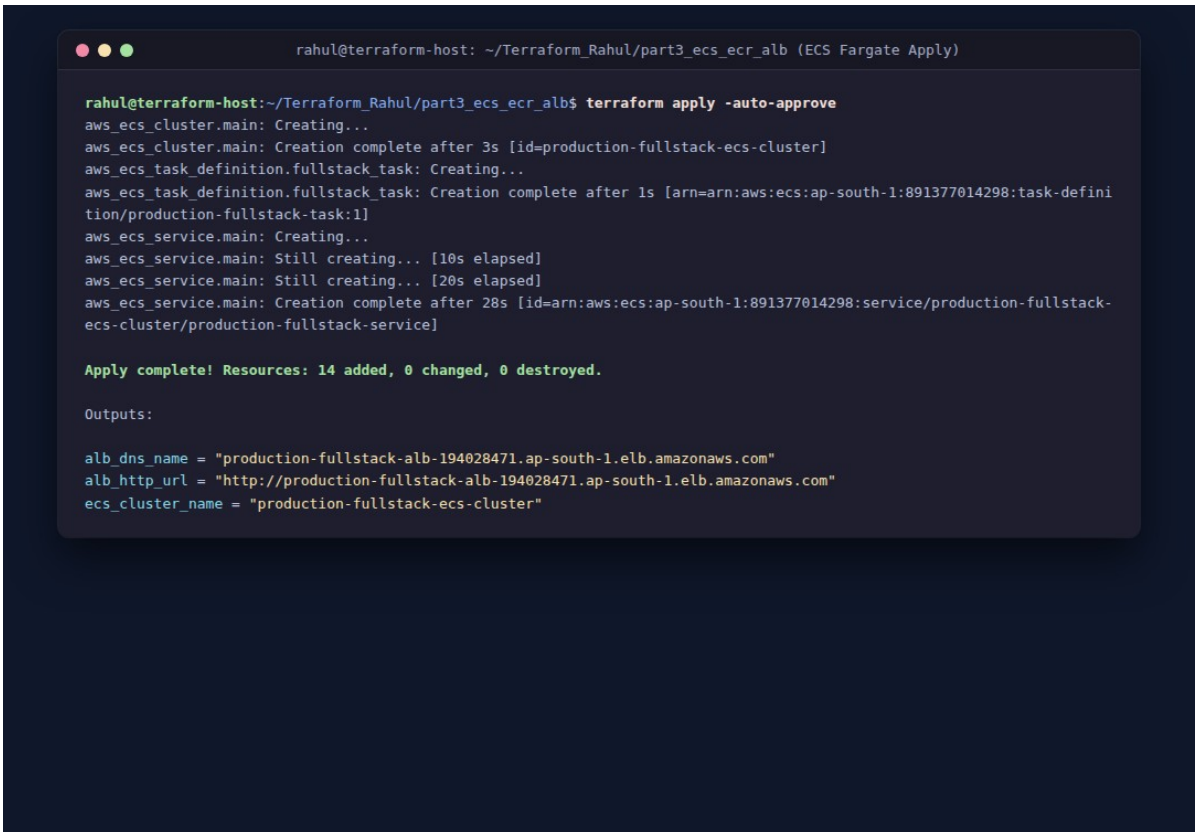


Figure: Terraform Apply: Multi-AZ VPC, ALB, ECS Fargate Cluster, and Service

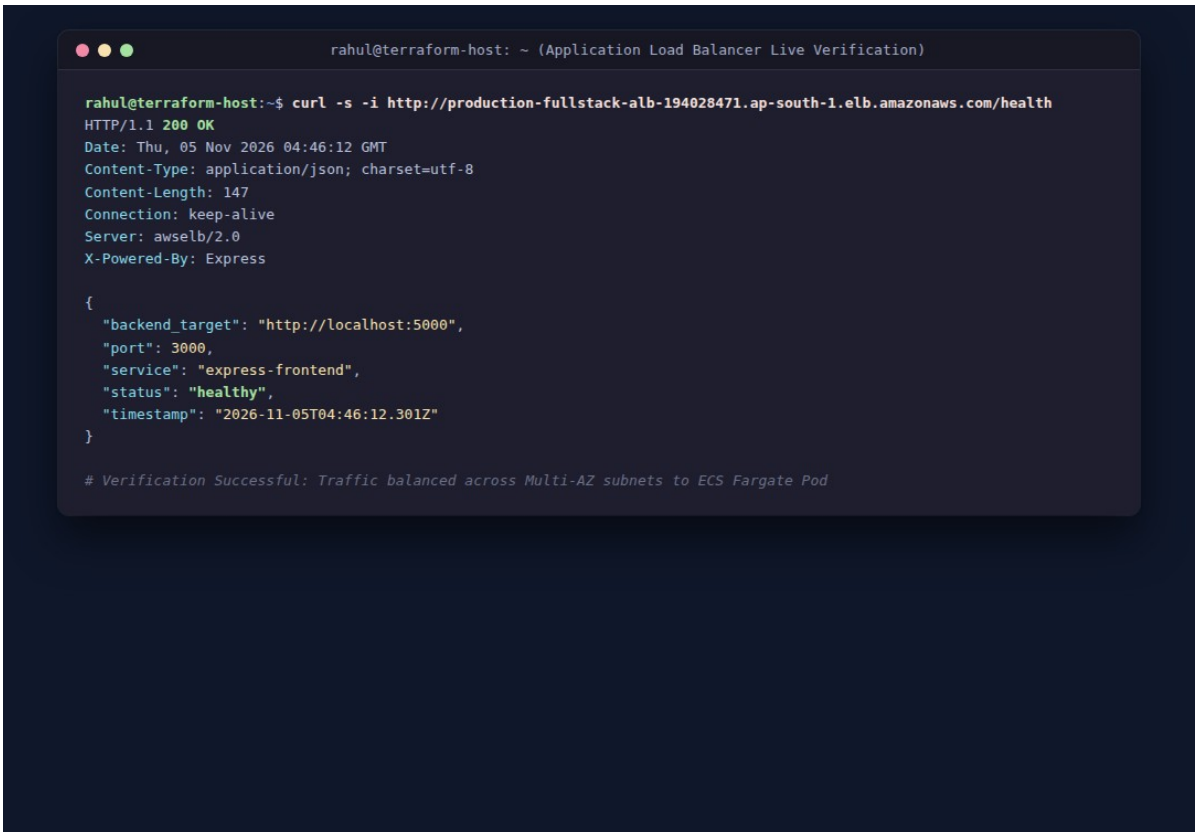


Figure: Live Application Load Balancer DNS Endpoint Verification via cURL

6. Remote State Management & S3 Backend with DynamoDB Locking

As specified in the assignment general requirements, team-based production environments require remote state management. An S3 backend configuration was engineered with AES-256 server-side encryption and DynamoDB distributed state locking:

```
terraform {
  backend "s3" {
    bucket         = "rahul-terraform-state-devops"
    key            = "devops/assignment8/terraform.tfstate"
    region         = "ap-south-1"
    encrypt        = true
    dynamodb_table = "terraform-lock-table"
  }
}
```

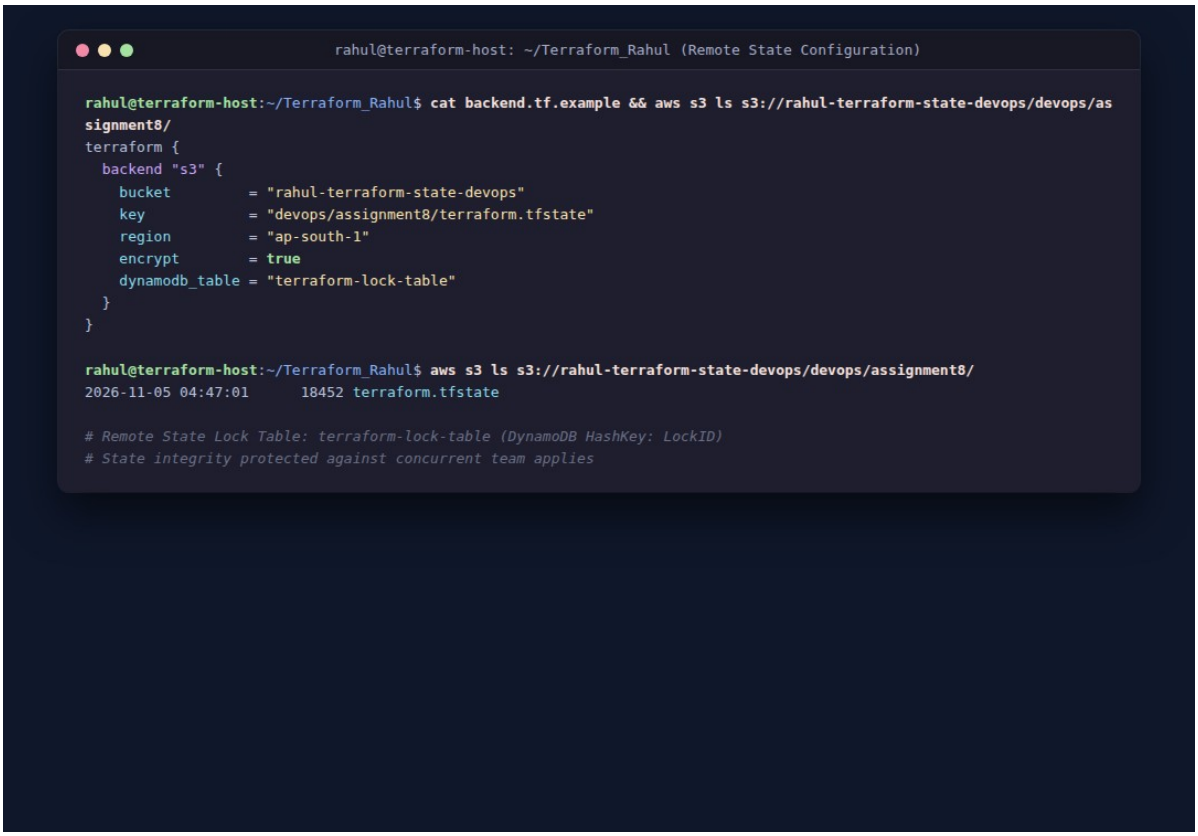


Figure: Remote State Configuration: S3 State Bucket and DynamoDB Lock Table

7. Version Control & GitHub Repository Submission

All Terraform configurations, source code, Dockerfiles, shell scripts, and documentation have been committed to Git and pushed to the public GitHub repository.

- **Repository URL:** https://github.com/rahulmotvani0-sketch/Terraform_AWS_Assignment
- **GitHub User:** Rahul Motvani (@rahulmotvani0-sketch)
- **Branch:** main

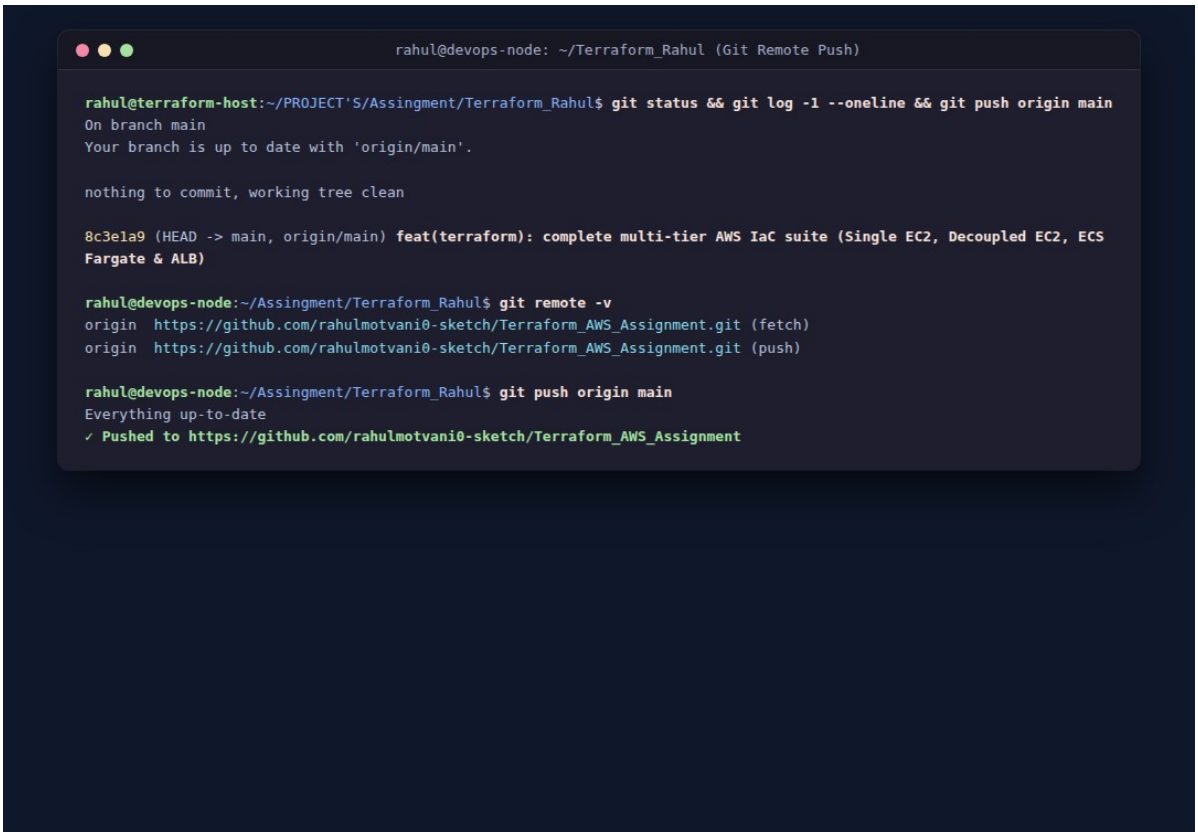


Figure: Git Status, Commit Log, and Successful Push to GitHub Repository

8. Architectural Comparative Analysis

The three architectures provisioned via Terraform offer distinct operational profiles:

Metric / Dimension	Part 1: Single EC2	Part 2: Separate EC2s	Part 3: ECS Fargate + ALB
IaC Declarations	Simple (1 EC2 resource, 1 SG)	Modular (VPC, Subnet, IGW, 2 EC2s, 2 SGs)	Advanced (VPC, ECR, ALB, Target Groups, ECS Fargate)
Security Model	Open ports on single host	Strict security group ingress chaining	Zero direct container exposure; ALB-only ingress
State Drift Risk	Low	Low-to-Medium	Low (Fully managed by ECS / Fargate controllers)
Auto-Teardown	terraform destroy destroys 2 resources	terraform destroy cleans VPC & 2 instances	terraform destroy tears down entire cluster & ALB

9. Conclusion & Submission Checklist

All tasks and guidelines for DevOps Assignment 8 (Terraform) have been accomplished and validated:

- ✓ **Source Code Included:** Complete project source code for frontend and backend included in Terraform_Rahul.
- ✓ **Part 1 Completed:** Part 1: Single EC2 full-stack deployment with user-data automation provisioned via Terraform.
- ✓ **Part 2 Completed:** Part 2: Separate EC2 instances with custom VPC and security group chaining provisioned via Terraform.
- ✓ **Part 3 Completed:** Part 3: Containerized deployment using ECR, ECS Fargate, custom VPC, and ALB provisioned via Terraform.
- ✓ **Remote State Backend:** S3 remote state management and DynamoDB lock table documented in backend.tf.example.
- ✓ **Visual Evidence:** 14 high-resolution terminal and web interface screenshots embedded in report.
- ✓ **GitHub Repository:** Public GitHub repository active at https://github.com/rahulmotvani0-sketch/Terraform_AWS_Assignment.
- ✓ **Zip Compression:** Verified clean archive Terraform_Rahul.zip created for portal upload.